VITyarthi | The Open Source Audit

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Rishi Raj	24BAI10118

Slot	Date of Submission
E-11	30-03-2026

Introduction

Open source software refers to software whose source code is freely available for anyone to view, modify, and distribute. Unlike proprietary software, open source encourages collaboration, transparency, and innovation. It forms the backbone of modern computing infrastructure, powering servers, mobile devices, cloud platforms, and the majority of the internet.

One of the most important examples of open source software is the Apache HTTP Server. Since its creation in 1995, Apache has played a critical role in shaping the web. It has consistently been one of the most widely used web servers in the world, powering a significant portion of websites globally.

This project presents a detailed audit of the Apache HTTP Server. It examines its origin, philosophy, licensing, ethical implications, technical footprint in Linux systems, and its role in the broader Free and Open Source Software (FOSS) ecosystem. The goal is to understand not just the technology, but also the principles and impact behind it.

Part A1: Origin and Philosophy

1. The Problem Apache Solved

In the early days of the web (1993–1995), the most widely used web server was NCSA HTTPd. However, development of this server slowed down significantly when key developers left to work at Netscape. This left web administrators without reliable updates or support.

At the same time, the web was growing rapidly, and there was a strong need for a stable, secure, and flexible web server. Commercial alternatives existed, but they were expensive and inaccessible to many users. There was no strong open source alternative available.

This gap created an opportunity for a community-driven solution.

2. The Origin Story

A group of web administrators, including Brian Behlendorf and Cliff Skolnick, began collaborating to improve the existing NCSA HTTPd server. They started sharing patches via email to fix bugs and add features.

Over time, these patches evolved into a complete server. The project became known as “Apache,” which is often humorously interpreted as “A Patchy Server.”

The first official release of Apache HTTP Server was in August 1995. The group of contributors formed what became known as the Apache Group, which later evolved into the Apache Software Foundation.

Apache quickly gained popularity due to its reliability, flexibility, and community-driven development model.

3. Why They Shared It

The creators of Apache were solving real problems they faced as web administrators. By sharing their solutions openly, they enabled others to benefit from their work.

There was no corporate ownership or profit motive initially. Instead, the project was guided by a philosophy of open collaboration. Contributors believed that software improves when more people can access and modify it.

This approach led to rapid innovation and widespread adoption. The guiding principle was simple: collaboration leads to better software.

Part A2: License Analysis

1. Apache License 2.0

The Apache HTTP Server is distributed under the Apache License 2.0, which is a permissive open source license. This means users are allowed to use, modify, and distribute the software with minimal restrictions.

Key features include:

- Freedom to use the software commercially
- Permission to modify and distribute code
- Ability to sublicense

However, users must:

- Include the original license and copyright notices

- Not misuse trademarks
- Provide proper attribution

The license also includes a patent grant, which protects users from patent-related legal issues.

2. The Four Freedoms

The Apache License supports all four essential freedoms of open source:

- Freedom 0: Run the program for any purpose
- Freedom 1: Study how the program works
- Freedom 2: Redistribute copies
- Freedom 3: Distribute modified versions

This confirms that Apache is truly free software in both practical and philosophical terms.

3. GPL vs MIT vs Apache

- **GPL License:** Requires that modified versions must also be open source (copyleft)
- **MIT License:** Very permissive, minimal restrictions
- **Apache License:** Permissive like MIT, but adds patent protection

The Apache License strikes a balance between freedom and legal protection, making it attractive for both developers and businesses.

4. Free Beer vs Freedom

“Free as in beer” means zero cost, while “free as in freedom” refers to control and rights over software.

Apache provides both:

- It is free to use
- It grants users full control over the software

This combination makes it powerful and widely adopted.

Part A3: Ethics of Open Source

1. Should All Software Be Open Source?

Arguments FOR:

- Transparency improves security
- Users gain control over software
- Prevents vendor lock-in
- Encourages learning and innovation

Arguments AGAINST:

- Some business models rely on proprietary software
- Trade secrets may need protection
- National security concerns
- Some systems require strict control

Not all software must be open source, but open source provides significant advantages in many scenarios.

2. Corporate Use of Free Labor

Many companies rely on open source:

- Red Hat builds business through support and contributes heavily
- Google and Meta contribute significantly to projects
- Amazon uses open source but contributes less proportionally

This raises ethical questions about whether companies should give back to the community.

3. Standing on the Shoulders of Giants

The concept means building upon previous work. Apache itself was built on NCSA HTTPd. Many modern applications like WordPress rely on Apache.

Open source allows developers to reuse and improve existing solutions, accelerating innovation.

Part B: Linux Footprint

1.Installation

Apache can be installed easily:

- Ubuntu/Debian:
  sudo apt install apache2

- RHEL/CentOS:
  sudo yum install httpd

It can also be compiled from source code.

2.Key Directories

Directory	Purpose
/etc/apache2	Configuration files
/var/www/html	Default website directory
/var/log/apache2	Logs
/usr/sbin/apache2	Executable
/usr/lib/apache2	Modules

3.User and Group

- Ubuntu: www-data
- RHEL: apache

This improves security by limiting permissions (principle of least privilege).

4.Service Management

sudo systemctl start apache2

sudo systemctl stop apache2

sudo systemctl restart apache2

sudo systemctl status apache2

5.Update Model

Updates are managed via package managers:

sudo apt update
sudo apt upgrade

Security patches are regularly released by the community.

Part C: The FOSS Ecosystem

1. Dependencies

Apache depends on:

- APR (Apache Portable Runtime)
- OpenSSL (security)
- PCRE (regex support)
- Zlib (compression)

2. What Apache has Enabled

Apache made possible:

- LAMP Stack (Linux, Apache, MySQL, PHP)
- WordPress (~40% of websites)
- Drupal, Joomla, MediaWiki

It is foundational to modern web development.

3. Community Structure and Governance

The Apache Software Foundation (ASF) governs the project.

Key features:

- Merit-based contributions
- Open mailing lists
- Global collaboration
- ApacheCon conferences
- 350+ projects

Part D: Open Source vs Proprietary

1.Comparison Table

Dimension	Apache HTTP Server	Microsoft IIS
Cost	Completely free to use, no licensing fees.	Requires Windows Server license (can be expensive)
Security	Transparent; vulnerabilities can be identified and patched by community quickly	Security handled internally by Microsoft; less transparency
Platform Support	Cross-platform (Linux, Windows, macOS, Unix)	Primarily Windows-only
Performance	Efficient and stable; widely optimized for various workloads	Optimized for Windows environments; strong integration with .NET
Support	Community forums, documentation, and third-party paid support	Official Microsoft support and enterprise support contracts
Community	Large global developer community contributing continuously	Controlled by a single company (Microsoft)
Market Share	~30% of websites globally	~15% of websites

2. Verdict

Apache HTTP Server is an excellent choice for most deployments. Its transparency allows better security auditing, and its community support is strong and reliable.

While proprietary solutions have their place, Apache offers unmatched flexibility and freedom. It is highly recommended for real-world applications.

Contributing to Apache is also encouraged, as it is a welcoming and impactful community.

Conclusion

This project highlights the importance of Apache HTTP Server in the history of open source software. It demonstrates how collaboration and shared knowledge can create powerful and reliable systems.

Through this audit, we learned about open source philosophy, licensing, ethics, and real-world implementation. Apache stands as a strong example of how open collaboration can shape the modern internet.

Shell Script Tasks

SCRIPT 1

CODE SCREENSHOTS:

```
script1.sh
#!/bin/bash
# ============================================================
# Script 1: System Identity Report
# Author: [Your Name] | Roll No: [Your Roll Number]
# Course: Open Source Software | Chosen Software: Apache HTTP Server
# Description: Displays a welcome screen with key system info
#              and the open-source license of the OS.
# ============================================================

# --- Variables ---
STUDENT_NAME="Rishi Raj"          # Replace with your actual name
SOFTWARE_CHOICE="Apache HTTP Server"  # Chosen OSS project

# --- Gather system information using command substitution $() ---
KERNEL=$(uname -r)                                # Linux kernel version
USER_NAME=$(whoami)                               # Currently logged-in user
HOME_DIR=$HOME                                    # Home directory of current user
UPTIME=$(uptime -p)                               # Human-readable uptime
CURRENT_DATE=$(date '+%d %B %Y, %H:%M:%S')    # Formatted current date and time

# Get the Linux distribution name from /etc/os-release file
# The 'grep' finds the PRETTY_NAME line, 'cut' extracts the value after '='
# 'tr -d' removes surrounding quotes from the value
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')

# --- Display the report as a formatted welcome screen ---
echo "================================================"
echo "   Open Source Audit — $STUDENT_NAME"
echo "   Auditing: $SOFTWARE_CHOICE"
echo "================================================"
echo ""
echo "  Linux Distribution : $DISTRO"
echo "  Kernel Version     : $KERNEL"
echo "  Logged-in User     : $USER_NAME"
echo "  Home Directory     : $HOME_DIR"
echo "  System Uptime      : $UPTIME"
echo "  Current Date/Time  : $CURRENT_DATE"
echo ""
echo "------------------------------------------------"
echo "  OS License Notice"
echo "------------------------------------------------"
# The Linux kernel (which powers this OS) is licensed under GPL v2.
# GPL v2 (GNU General Public License version 2) grants users the freedom
# to run, study, modify, and redistribute the software, provided that
# any modified versions are also distributed under the same GPL v2 terms.
echo "  The Linux kernel running this system is"
echo "  licensed under the GNU General Public"
echo "  License version 2 (GPL v2)."
echo ""
echo "  GPL v2 guarantees: freedom to run, study,"
echo "  modify, and share this OS and its source."
echo "================================================"
```

EXECTUION SCREENSHOTS:

```
rishi@Rishi:~/OSS-Project-RishiRaj$ ./script1_system_identity.sh
================================================
   Open Source Audit — Rishi Raj
   Auditing: Apache HTTP Server
================================================

  Linux Distribution : Ubuntu 24.04.3 LTS
  Kernel Version     : 6.6.87.2-microsoft-standard-WSL2
  Logged-in User     : rishi
  Home Directory     : /home/rishi
  System Uptime      : up 10 minutes
  Current Date/Time  : 30 March 2026, 16:26:02

------------------------------------------------
  OS License Notice
------------------------------------------------
  The Linux kernel running this system is
  licensed under the GNU General Public
  License version 2 (GPL v2).

  GPL v2 guarantees: freedom to run, study,
  modify, and share this OS and its source.
================================================
```

SCRIPT 2:

CODE SCREENSHOTS:

```
script2.sh
# Script 2: FOSS Package Inspector
# Author: [Your Name] | Roll No: [Your Roll Number]
# Course: Open Source Software | Chosen Software: Apache HTTP Server
# Description: Checks if a FOSS package is installed, shows its
#              version/license, and prints a philosophy note
#              about it using a case statement.
# ==========================================================

# --- Package to inspect ---
# Apache HTTP Server is called 'apache2' on Debian/Ubuntu
# and 'httpd' on RHEL/CentOS/Fedora systems
PACKAGE="apache2"   # Change to 'httpd' if you are on CentOS/RHEL/Fedora

echo "================================================"
echo "  FOSS Package Inspector"
echo "================================================"
echo ""

# --- Detect the package manager available on this system ---
# This makes the script work on both Debian-based and RPM-based distros
if command -v dpkg &>/dev/null; then
    PKG_MANAGER="dpkg"   # Debian / Ubuntu
elif command -v rpm &>/dev/null; then
    PKG_MANAGER="rpm"    # RHEL / CentOS / Fedora
else
    echo "ERROR: No recognised package manager (dpkg or rpm) found."
    exit 1
fi

echo "  Package Manager Detected: $PKG_MANAGER"
echo ""

# --- if-then-else: Check whether the package is installed ---
if [ "$PKG_MANAGER" = "dpkg" ]; then
    # dpkg -l lists packages; grep searches for our package name
    # &>/dev/null suppresses output — we only care about exit status (0 = found)
    dpkg -l "$PACKAGE" &>/dev/null
    INSTALLED=$?   # $? holds exit code of last command (0 = success)
else
    rpm -q "$PACKAGE" &>/dev/null
    INSTALLED=$?
fi

if [ $INSTALLED -eq 0 ]; then
    echo "  STATUS: $PACKAGE is INSTALLED on this system."
    echo ""
    echo "  --- Package Details ---"

    # Extract and display Version, License, and Summary fields
    if [ "$PKG_MANAGER" = "dpkg" ]; then
        # dpkg-query gives detailed info; grep with -E allows multiple patterns
        dpkg-query -s "$PACKAGE" | grep -E 'Version|Homepage|Description'
    else
        rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary'
    fi
else
    echo "  STATUS: $PACKAGE is NOT installed on this system."
    echo ""
    echo "  To install Apache on Ubuntu/Debian, run:"
    echo "    sudo apt update && sudo apt install apache2"
    echo ""
```

```
echo ""
echo "----------------------------------------------"
echo "  Open Source Philosophy Notes"
echo "----------------------------------------------"

# --- case statement: Print a philosophy note for each known FOSS package ---
# The case statement matches the value of $PACKAGE against several patterns
case $PACKAGE in
    apache2 | httpd)
        echo "  Apache HTTP Server:"
        echo "  Licensed under Apache License 2.0."
        echo "  Powers ~30% of all websites globally."
        echo "  It proved that collaborative, open development"
        echo "  could outperform commercial software at scale."
        ;;
    mysql | mariadb)
        echo "  MySQL / MariaDB:"
        echo "  A dual-licensed database (GPL v2 + Commercial)."
        echo "  MariaDB was forked from MySQL by the original"
        echo "  creator after Oracle acquired Sun Microsystems —"
        echo "  a real-world example of forking to preserve freedom."
        ;;
    firefox)
        echo "  Mozilla Firefox:"
        echo "  Licensed under MPL 2.0."
        echo "  A nonprofit browser built to keep the web open."
        echo "  Exists to prevent any single company from"
        echo "  controlling how the internet is experienced."
        ;;
    vlc)
        echo "  VLC Media Player:"
        echo "  Licensed under LGPL/GPL."
        echo "  Started by students at École Centrale Paris."
        echo "  Plays virtually every media format — built freely,"
        echo "  shared freely, used by hundreds of millions."
        ;;
    python3 | python)
        echo "  Python:"
        echo "  Licensed under PSF License."
        echo "  Shaped entirely by community consensus."
        echo "  Demonstrates how open governance can produce"
        echo "  a language trusted by science, education, and industry."
        ;;
    git)
        echo "  Git:"
        echo "  Licensed under GPL v2."
        echo "  Created by Linus Torvalds in 2005 when the"
        echo "  proprietary BitKeeper license was revoked."
        echo "  Now the backbone of nearly all software development."
        ;;
    *)
        # Default case for any unrecognised package
        echo "  $PACKAGE: An open-source tool that embodies"
        echo "  the principle that knowledge shared is knowledge"
        echo "  multiplied. Freedom to study and modify makes"
        echo "  software safer and more trustworthy for everyone."
        ;;
esac

echo "==============================================="
```

EXECTUION SCREENSHOTS:

```
rishi@Rishi:~/OSS-Project-RishiRaj$ ./script2_package_inspector.sh
===============================================
  FOSS Package Inspector
===============================================

  Package Manager Detected: dpkg

  STATUS: apache2 is INSTALLED on this system.

  --- Package Details ---
Version: 2.4.58-1ubuntu8.11
Description: Apache HTTP Server
Homepage: https://httpd.apache.org/

-----------------------------------------------
  Open Source Philosophy Notes
-----------------------------------------------
  Apache HTTP Server:
  Licensed under Apache License 2.0.
  Powers ~30% of all websites globally.
  It proved that collaborative, open development
  could outperform commercial software at scale.
-----------------------------------------------
```

SCRIPT 3

CODE SCREENSHOTS:

```bash
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Author: [Your Name] | Roll No: [Your Roll Number]
# Course: Open Source Software | Chosen Software: Apache HTTP Server
# Description: Loops through key system directories and reports
#              permissions, owner, and disk usage for each.
#              Also checks Apache-specific config directories.
# ============================================================

# --- List of standard system directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "================================================"
echo "  Disk and Permission Auditor"
echo "================================================"
echo ""
echo "  --- Standard System Directory Audit ---"
echo ""

# --- for loop: Iterate over each directory in the DIRS array ---
for DIR in "${DIRS[@]}"; do

    # Check if the directory actually exists before trying to inspect it
    if [ -d "$DIR" ]; then

        # ls -ld lists the directory itself (not its contents)
        # awk '{print $1, $3, $4}' extracts: permissions, owner-user, owner-group
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

        # du -sh gives human-readable size of the directory
        # 2>/dev/null suppresses permission-denied errors
        # cut -f1 takes only the size column (before the tab)
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        echo "  Directory : $DIR"
        echo "  Perms/Owner: $PERMS"
        echo "  Size      : $SIZE"
        echo "  ----------------------------------------------"
    else
        echo "  $DIR does NOT exist on this system."
        echo "  ----------------------------------------------"
    fi
done

echo ""
echo "  --- Apache HTTP Server Directory Audit ---"
echo ""

# Apache-specific directories to check
# These paths are standard on Debian/Ubuntu (apache2)
# On CentOS/RHEL: config is /etc/httpd, logs at /var/log/httpd
APACHE_DIRS=(
    "/etc/apache2"          # Main Apache config directory (Debian/Ubuntu)
    "/etc/httpd"            # Main Apache config directory (RHEL/CentOS)
    "/var/log/apache2"      # Apache log files (Debian/Ubuntu)
    "/var/log/httpd"        # Apache log files (RHEL/CentOS)
    "/var/www/html"         # Default web document root
    "/usr/sbin/apache2"     # Apache binary (Debian/Ubuntu)
    "/usr/sbin/httpd"       # Apache binary (RHEL/CentOS)
```

```bash
)

# Counter to track how many Apache directories were found
FOUND=0

# Loop through Apache-specific directories
for ADIR in "${APACHE_DIRS[@]}"; do

    # Check if this path exists (as a file OR directory)
    if [ -e "$ADIR" ]; then
        FOUND=$((FOUND + 1))   # Increment counter using arithmetic expansion

        # ls -ld works for both files and directories
        APERMS=$(ls -ld "$ADIR" | awk '{print $1, $3, $4}')
        ASIZE=$(du -sh "$ADIR" 2>/dev/null | cut -f1)

        echo "  [FOUND] $ADIR"
        echo "  Perms/Owner: $APERMS"
        echo "  Size      : $ASIZE"
        echo "  ----------------------------------------------"
    fi
done

# Inform the user if no Apache directories were found at all
if [ $FOUND -eq 0 ]; then
    echo "  No Apache directories found."
    echo "  Apache may not be installed on this system."
    echo "  Install with: sudo apt install apache2"
    echo "              or: sudo dnf install httpd"
fi

echo ""
echo "  Audit complete."
echo "================================================"
```

EXECTUION SCREENSHOTS:

```
rishi@Rishi:~/OSS-Project-RishiRaj$ ./script3_disk_auditor.sh
==================================================
  Disk and Permission Auditor
==================================================

  --- Standard System Directory Audit ---

  Directory : /etc
  Perms/Owner: drwxr-xr-x root root
  Size      : 9.8M
  --------------------------------------------------
  Directory : /var/log
  Perms/Owner: drwxrwxr-x root syslog
  Size      : 188M
  --------------------------------------------------
  Directory : /home
  Perms/Owner: drwxr-xr-x root root
  Size      : 5.5G
  --------------------------------------------------
  Directory : /usr/bin
  Perms/Owner: drwxr-xr-x root root
  Size      : 133M
  --------------------------------------------------
  Directory : /tmp
  Perms/Owner: drwxrwxrwt root root
  Size      : 64K
  --------------------------------------------------

  --- Apache HTTP Server Directory Audit ---

  [FOUND] /etc/apache2
  Perms/Owner: drwxr-xr-x root root
  Size      : 688K
  --------------------------------------------------
  [FOUND] /var/log/apache2
  Perms/Owner: drwxr-x--- root adm
  Size      : 8.0K
  --------------------------------------------------
  [FOUND] /var/www/html
  Perms/Owner: drwxr-xr-x root root
  Size      : 16K
  --------------------------------------------------
  [FOUND] /usr/sbin/apache2
  Perms/Owner: -rwxr-xr-x root root
  Size      : 740K
  --------------------------------------------------

  Audit complete.
==================================================
```

SCRIPT 4

CODE SCREENSHOTS:

```
script4.sh
#!/bin/bash
# ==========================================================
# Script 4: Log File Analyzer
# Author: [Your Name] | Roll No: [Your Roll Number]
# Course: Open Source Software | Chosen Software: Apache HTTP Server
# Description: Reads a log file line by line, counts occurrences
#              of a keyword, and prints matching lines.
#              Designed to work with Apache access/error logs.
# Usage: ./script4_log_analyzer.sh /var/log/apache2/error.log [keyword]
# ==========================================================

# $1 is the first command-line argument — the log file path
LOGFILE=$1

# $2 is the optional second argument — keyword to search for
# If not provided, default to "error" using the := default syntax
KEYWORD=${2:-"error"}

# Counter variable to track how many matching lines are found
COUNT=0

# Array to store last 5 matching lines for display at the end
LAST_MATCHES=()

echo "==============================================="
echo "  Apache Log File Analyzer"
echo "==============================================="
echo ""

# --- Input validation: Check if the user provided a file path ---
if [ -z "$LOGFILE" ]; then
    echo "  ERROR: No log file specified."
    echo ""
    echo "  Usage: $0 <logfile> [keyword]"
    echo ""
    echo "  Apache log file locations:"
    echo "    Ubuntu/Debian : /var/log/apache2/error.log"
    echo "                    /var/log/apache2/access.log"
    echo "    CentOS/RHEL   : /var/log/httpd/error_log"
    echo "                    /var/log/httpd/access_log"
    exit 1
fi

# --- Check if the specified file actually exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "  ERROR: File '$LOGFILE' not found."
    echo ""

    # Do-while style retry: ask the user to try a fallback log path
    # This mimics a retry loop — bash doesn't have do-while natively,
    # but we simulate it with a while loop that runs at least once
    RETRY=1
    while [ $RETRY -eq 1 ]; do
        echo "  Would you like to try the default Apache log path?"
        echo "  Press Enter to try /var/log/apache2/error.log, or type 'n' to exit:"
        read -r USER_INPUT

        if [ "$USER_INPUT" = "n" ]; then
            echo "  Exiting. Please provide a valid log file path."
            exit 1
        else
```

```
script.sh
if [ ! -f "$LOGFILE" ]; then
    while [ $RETRY -eq 1 ]; do
        else
            # Try the default fallback path
            FALLBACK="/var/log/apache2/error.log"
            if [ -f "$FALLBACK" ]; then
                echo "  Using fallback: $FALLBACK"
                LOGFILE="$FALLBACK"
                RETRY=0   # Exit the retry loop
            else
                # Fallback also not found — stop retrying
                echo "  Fallback log not found either. Is Apache installed?"
                echo "  Install with: sudo apt install apache2"
                exit 1
            fi
        fi
    done
fi

# --- Check if the file is empty ---
if [ ! -s "$LOGFILE" ]; then
    echo "  WARNING: Log file '$LOGFILE' is empty."
    echo "  Apache may not have written any entries yet."
    echo "  Try accessing your web server and re-run this script."
    exit 0
fi

echo "  Log File : $LOGFILE"
echo "  Keyword  : '$KEYWORD' (case-insensitive)"
echo ""
echo "  Scanning..."
echo ""

# --- while read loop: Read the log file one line at a time ---
# IFS= prevents leading/trailing whitespace from being stripped
# -r prevents backslash interpretation (raw mode)
while IFS= read -r LINE; do

    # if-then: Check if this line contains the keyword (case-insensitive)
    # grep -i = case insensitive, -q = quiet (no output, just exit code)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment counter with arithmetic expansion

        # Store in LAST_MATCHES array (keeps growing, we'll use tail logic below)
        LAST_MATCHES+=("$LINE")
    fi

done < "$LOGFILE"   # Redirect the file into the while loop as input

# --- Display results ---
echo "----------------------------------------------"
echo "  SCAN RESULTS"
echo "----------------------------------------------"
echo "  Keyword '$KEYWORD' found $COUNT time(s) in:"
echo "  $LOGFILE"
echo ""

# Show the last 5 matching lines if any were found
if [ $COUNT -gt 0 ]; then
    echo "  --- Last 5 Matching Lines ---"
```

```
echo "  Log File : $LOGFILE"
echo "  Keyword  : '$KEYWORD' (case-insensitive)"
echo ""
echo "  Scanning..."
echo ""

# --- while read loop: Read the log file one line at a time ---
# IFS= prevents leading/trailing whitespace from being stripped
# -r prevents backslash interpretation (raw mode)
while IFS= read -r LINE; do

    # if-then: Check if this line contains the keyword (case-insensitive)
    # grep -i = case insensitive, -q = quiet (no output, just exit code)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment counter with arithmetic expansion

        # Store in LAST_MATCHES array (keeps growing, we'll use tail logic below)
        LAST_MATCHES+=("$LINE")
    fi

done < "$LOGFILE"   # Redirect the file into the while loop as input

# --- Display results ---
echo "----------------------------------------------"
echo "  SCAN RESULTS"
echo "----------------------------------------------"
echo "  Keyword '$KEYWORD' found $COUNT time(s) in:"
echo "  $LOGFILE"
echo ""

# Show the last 5 matching lines if any were found
if [ $COUNT -gt 0 ]; then
    echo "  --- Last 5 Matching Lines ---"
    echo ""

    # Calculate starting index to get the last 5 entries from the array
    TOTAL=${#LAST_MATCHES[@]}   # Total number of matching lines

    # If fewer than 5 matches, start from 0; otherwise start from (total - 5)
    if [ $TOTAL -le 5 ]; then
        START=0
    else
        START=$((TOTAL - 5))
    fi

    # Loop from START to end of array and print each line
    for (( i=START; i<TOTAL; i++ )); do
        echo "  [Match $((i+1))] ${LAST_MATCHES[$i]}"
        echo ""
    done
else
    echo "  No lines containing '$KEYWORD' were found."
    echo "  Try a different keyword like 'warn', 'crit', or 'GET'."
fi

echo "==============================================="
```

EXECTUION SCREENSHOTS:

```
rishi@Rishi:~/OSS-Project-RishiRaj$ ./script4_log_analyzer.sh
==================================================
  Apache Log File Analyzer
==================================================

  ERROR: No log file specified.

  Usage: ./script4_log_analyzer.sh <logfile> [keyword]

  Apache log file locations:
    Ubuntu/Debian : /var/log/apache2/error.log
                    /var/log/apache2/access.log
    CentOS/RHEL   : /var/log/httpd/error_log
                    /var/log/httpd/access_log
```

SCRIPT 5

CODE SCREENSHOTS:

```
script5.sh
#!/bin/bash
# ==========================================================
# Script 5: Open Source Manifesto Generator
# Author: [Your Name] | Roll No: [Your Roll Number]
# Course: Open Source Software | Chosen Software: Apache HTTP Server
# Description: Asks the user three interactive questions and
#              composes a personalised open-source philosophy
#              statement, saving it to a .txt file.
# ==========================================================

# --- Alias concept (demonstrated via comment) ---
# In a real shell session, you might define an alias like:
#   alias makemanifesto='bash script5_manifesto_generator.sh'
# This would let you run the script by just typing: makemanifesto
# Aliases are session-level shortcuts — they live in ~/.bashrc
# We don't define the alias inside the script (it wouldn't persist),
# but we document the concept here as required.

echo "==============================================="
echo "  Open Source Manifesto Generator"
echo "  Inspired by: Apache HTTP Server"
echo "==============================================="
echo ""
echo "  Answer three questions below."
echo "  Your answers will be woven into a personal"
echo "  open-source philosophy statement."
echo ""

# --- read: Prompt user for interactive input ---
# -p flag lets us put the prompt on the same line as the cursor
read -p "  1. Name one open-source tool you use every day: " TOOL
echo ""

read -p "  2. In one word, what does 'freedom' mean to you?: " FREEDOM
echo ""

read -p "  3. Name one thing you would build and share freely: " BUILD
echo ""

# --- Validate that the user actually entered something ---
# -z checks if a string is empty
if [ -z "$TOOL" ] || [ -z "$FREEDOM" ] || [ -z "$BUILD" ]; then
    echo "  ERROR: All three answers are required."
    echo "  Please re-run the script and provide all inputs."
    exit 1
fi

# --- Date and output filename ---
DATE=$(date '+%d %B %Y')              # e.g. 15 June 2025
TIME=$(date '+%H:%M:%S')              # e.g. 14:32:07
OUTPUT="manifesto_$(whoami).txt"      # Filename includes the current user's name

echo ""
echo "  Generating your manifesto..."
echo ""

# --- String concatenation: Build the manifesto paragraph ---
# Each echo >> appends a new line to the output file
# > would overwrite; >> appends — we use > for the first line, >> for rest

# First line: overwrite (create/reset) the file
```

```
script5.sh
# First line: overwrite (create/reset) the file
echo "==============================================" > "$OUTPUT"

# Append the rest of the manifesto using >>
echo "  OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo "  Generated on: $DATE at $TIME" >> "$OUTPUT"
echo "  Author: $(whoami)" >> "$OUTPUT"
echo "==============================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"

# The manifesto paragraph uses string concatenation — the variables
# $TOOL, $FREEDOM, and $BUILD are embedded directly into the text
echo "  Every day, I rely on $TOOL — a tool I did not pay" >> "$OUTPUT"
echo "  for, did not ask permission to use, and can study" >> "$OUTPUT"
echo "  down to its last line of code. To me, freedom means" >> "$OUTPUT"
echo "  $FREEDOM. That is what open source gives the world:" >> "$OUTPUT"
echo "  not just free software, but the freedom to understand," >> "$OUTPUT"
echo "  improve, and share the tools we all depend on." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  One day, I want to build $BUILD and release it" >> "$OUTPUT"
echo "  openly — because knowledge grows when it is shared," >> "$OUTPUT"
echo "  and the most powerful thing a developer can do is" >> "$OUTPUT"
echo "  make someone else's work a little easier." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  Apache HTTP Server stands as proof of this: built" >> "$OUTPUT"
echo "  freely, shared freely, and trusted to serve a third" >> "$OUTPUT"
echo "  of the entire internet. That is what open source" >> "$OUTPUT"
echo "  looks like when a community chooses transparency" >> "$OUTPUT"
echo "  over profit." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  — Written under the values of the Apache License 2.0" >> "$OUTPUT"
echo "==============================================" >> "$OUTPUT"

# Confirm to the user that the file was created
echo "  Manifesto saved to: $OUTPUT"
echo ""
echo "----------------------------------------------"

# Display the saved file using cat
cat "$OUTPUT"

echo ""
echo "==============================================="
echo "  Done. Your manifesto is ready to submit."
echo "==============================================="
```

EXECTUION SCREENSHOTS:

```
rishi@Rishi:~/OSS-Project-RishiRaj$ ./script5_manifesto.sh
==================================================
  Open Source Manifesto Generator
  Inspired by: Apache HTTP Server
==================================================

  Answer three questions below.
  Your answers will be woven into a personal
  open-source philosophy statement.

  1. Name one open-source tool you use every day: GITHUB

  2. In one word, what does 'freedom' mean to you?: PEACE

  3. Name one thing you would build and share freely: GITHUB AI

  Generating your manifesto...

  Manifesto saved to: manifesto_rishi.txt

----------------------------------------------
==============================================
  OPEN SOURCE MANIFESTO
  Generated on: 30 March 2026 at 16:27:51
  Author: rishi
==============================================

  Every day, I rely on GITHUB — a tool I did not pay
  for, did not ask permission to use, and can study
  down to its last line of code. To me, freedom means
  PEACE. That is what open source gives the world:
  not just free software, but the freedom to understand,
  improve, and share the tools we all depend on.

  One day, I want to build GITHUB AI and release it
  openly — because knowledge grows when it is shared,
  and the most powerful thing a developer can do is
  make someone else's work a little easier.

  Apache HTTP Server stands as proof of this: built
  freely, shared freely, and trusted to serve a third
  of the entire internet. That is what open source
  looks like when a community chooses transparency
  over profit.

  — Written under the values of the Apache License 2.0
==============================================

==============================================
  Done. Your manifesto is ready to submit.
==============================================
```